INTRODUCTION TO DATA SCIENCE · AUDITORY EXERCISES

Exercise 1 · Part 1: Data and Classical Models

Data Gathering and Exploratory Analysis

Ana Todorovska
Dimitar Peshevski
Marija Stojcheva
Viktor Kostadinoski

Faculty of Computer Science and Engineering
Ss. Cyril and Methodius University in Skopje

Winter Semester
2026/2027

Serving the Stored Site

PYTHON

```python
import json
from http.server import SimpleHTTPRequestHandler
from urllib.parse import parse_qs, urlparse
STATS = json.loads(open("data/api/listing-stats.json", encoding="utf-8").read())
class Portal(SimpleHTTPRequestHandler):
    extensions_map = {**SimpleHTTPRequestHandler.extensions_map, ".html": "text/html; charset=utf-8"}
    def do_GET(self) -> None:
        url = urlparse(self.path)
        if url.path != "/api/listing-stats":
            return super().do_GET()
        page = int(parse_qs(url.query).get("page", ["1"])[0])
        body = {"total": len(STATS), "page": page, "per_page": 50, "results": STATS[(page - 1) * 50:page * 50]}
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(body).encode())
    def log_message(self, format: str, *args: object) -> None:
        pass
```

Starting the Local Server

PYTHON

```python
import threading
from functools import partial
from http.server import ThreadingHTTPServer

server = ThreadingHTTPServer(("127.0.0.1", 0), partial(Portal, directory="data/site"))
threading.Thread(target=server.serve_forever, daemon=True).start()
BASE = f"http://127.0.0.1:{server.server_port}/"
```

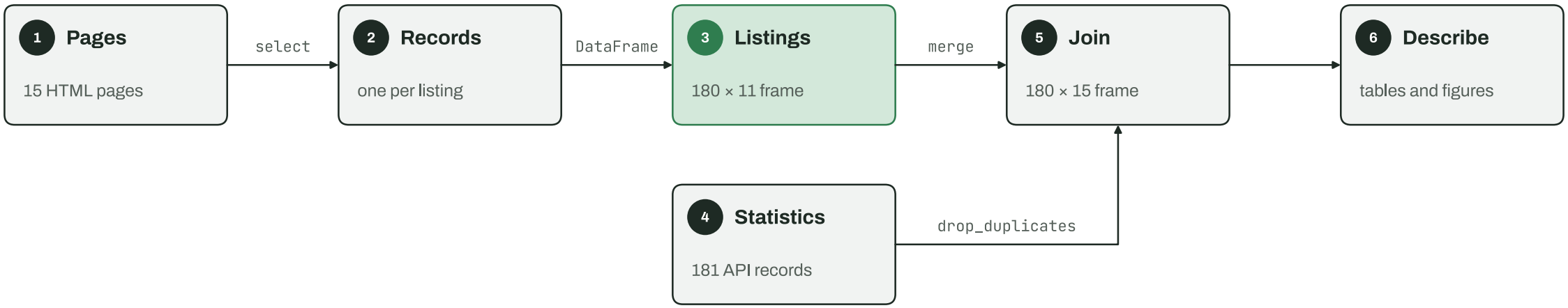
The server answers on the loopback address only, and port 0 lets the system choose a free port. Every request in this deck goes to `BASE`.

PART 1 · EXERCISES 1-6

Data and Classical Models

Gathering · Inference · Preparation · Regression · Classification · Ensembles

The Worked Example



The session collects the apartments for sale on Skopje Flats, a stored listings portal served from this machine, and describes them without modelling them. The listings are synthetic. The highlighted step, the listings frame, is the table the rest of the session builds on.

01

Collecting Data

Requests, selectors, pagination, a rendered page, an endpoint, and a join

Requests and Responses



A response carries a status code, headers, and a body. The status code is read first, because a body that arrives with an error code is an error page rather than the data. The highlighted step, the response, is what the next two slides inspect.

Sending a Request

PYTHON

```python
import requests

response = requests.get(BASE + "listings/page-1.html", timeout=10)
response.raise_for_status()
response.status_code, response.headers["Content-Type"]
```

OUTPUT

```
(200, 'text/html; charset=utf-8')
```

Status Codes

Code	Meaning
`200`	The request succeeded, and the body is the resource.
`301`, `302`	The resource moved, and `requests` follows the new address.
`404`	The resource does not exist, so the body is an error page.
`429`	The client sent too many requests and should wait.
`500`	The server failed, and a later retry may succeed.

`response.ok` is true below 400, and `raise_for_status()` raises an exception from 400 upward.

Parsing a Page

PYTHON

```python
from pathlib import Path

import pandas as pd
from bs4 import BeautifulSoup, Tag

html = Path("data/site/listings/page-1.html").read_text(encoding="utf-8")
soup = BeautifulSoup(html, "lxml")
cards = soup.select("article.listing")
len(cards), cards[0]["data-id"]
```

OUTPUT

```
(12, '2563')
```

Four Kinds of Selector

PYTHON

```
{
    "tag": len(soup.select("article")),
    "class": len(soup.select(".elevator")),
    "identifier": len(soup.select("#listing-2563")),
    "descendant": len(soup.select("ul.features li")),
    "no match": len(soup.select(".balcony")),
}
```

OUTPUT

```
{'tag': 12, 'class': 12, 'identifier': 1, 'descendant': 35, 'no match': 0}
```

An Empty Match Is a Normal Outcome

PYTHON

```
soup.select(".balcony"), soup.select_one(".balcony")
```

OUTPUT

```
([], None)
```

`select()` returns an empty list and `select_one()` returns `None`, so the code checks an element before it reads one that may be absent.

Numbers Inside a Sentence

The Price

PYTHON

```python
import re

PRICE = re.compile(r"(\d[\d ]*) (EUR|MKD)")
card = cards[0]
text = card.select_one(".price").get_text()
PRICE.search(text).groups()
```

OUTPUT

```
('72 000', 'EUR')
```

The Summary

PYTHON

```python
SUMMARY = re.compile(
    r"([\d.]+) m² apartment on floor (\d+)"
    r" of (\d+), built in (\d{4})")
text = card.select_one(".summary").get_text()
SUMMARY.search(text).groups()
```

OUTPUT

```
('73', '6', '8', '1958')
```

Required and Optional Fields

PYTHON

```python
def required_text(card: Tag, selector: str) -> str:
    element = card.select_one(selector)
    if element is None:
        raise ValueError(f"Listing {card['data-id']} has no {selector} element.")
    return element.get_text(strip=True)
def optional_text(card: Tag, selector: str) -> str | None:
    element = card.select_one(selector)
    return element.get_text(strip=True) if element else None
def captures(pattern: re.Pattern[str], text: str) -> tuple[str, ...]:
    match = pattern.search(text)
    if match is None:
        raise ValueError(f"{text!r} does not match {pattern.pattern!r}.")
    return match.groups()
```

The absence of a required field means the page has changed, so the parser raises an error. An absent optional field becomes a missing value.

One Record per Element

PYTHON

```python
def parse_listing(card: Tag) -> dict[str, object]:
    amount, currency = captures(PRICE, required_text(card, ".price"))
    area, floor, floors, year = captures(SUMMARY, required_text(card, ".summary"))
    elevator = optional_text(card, ".elevator")
    return {"listing_id": int(card["data-id"]),
            "municipality": required_text(card, ".title").split(" in ")[-1],
            "price": int(amount.replace(" ", "")), "currency": currency,
            "area": float(area), "rooms": int(required_text(card, ".rooms").split()[0]),
            "floor": int(floor), "floors": int(floors), "year_built": int(year),
            "condition": optional_text(card, ".condition"),
            "elevator": None if elevator is None else elevator.endswith("yes")}
```

NOTEBOOK · SECTIONS 1 TO 3

Requests, Selectors, and Records

Run the cells from `## Setup` to `### 3. One Record per Element`.

Pagination

PYTHON

```python
records, path = [], Path("data/site/listings/page-1.html")
while path:
    page = BeautifulSoup(path.read_text(encoding="utf-8"), "lxml")
    records += [parse_listing(card) for card in page.select("article.listing")]
    link = page.select_one("a.next")
    path = path.parent / link["href"] if link else None
listings = pd.DataFrame(records)
listings.shape
```

OUTPUT

```
(180, 11)
```

The notebook runs the same loop over HTTP, with `requests.get` and `urljoin` in place of the file paths.

The Listings Frame


```python
listings.iloc[:, :6].head()
```

	listing_id	municipality	price	currency	area	rooms
0	2563	Гази Баба	72000	EUR	73.0	3
1	4159	Чаир	42000	EUR	38.0	1
2	6002	Аеродром	55500	EUR	41.4	1
3	7035	Карпош	90500	EUR	75.0	3
4	1836	Карпош	206500	EUR	110.0	4

A Page That Renders Itself

PYTHON

```python
from playwright.async_api import async_playwright
async with async_playwright() as playwright:
    browser = await playwright.chromium.launch()
    tab = await browser.new_page()
    await tab.goto(BASE + "market-index.html")
    await tab.wait_for_selector("#index td")
    rendered = await tab.content()
    await browser.close()
len(BeautifulSoup(rendered, "lxml").select("#index tr:has(td)"))
```

OUTPUT

```
8
```

The page builds its table with a script, so `requests` receives an empty `#index`. The browser runs the script and waits for a cell before it reads the markup.

A Paged Endpoint

PYTHON

```python
collected, page_number = [], 1
while True:
    body = requests.get(BASE + "api/listing-stats", params={"page": page_number},
                        timeout=10).json()
    collected += body["results"]
    if len(collected) >= body["total"] or not body["results"]:
        break
    page_number += 1

len(collected), body["total"]
```

OUTPUT

```
(181, 181)
```

The client stops when its records account for the reported total, and also on an empty page.

A Join Declared One to One

PYTHON

```python
import json

records = json.loads(Path("data/api/listing-stats.json").read_text(encoding="utf-8"))
stats = pd.DataFrame(records)
try:
    listings.merge(stats, on="listing_id", validate="one_to_one")
except pd.errors.MergeError as error:
    print(str(error).splitlines()[0])
```

OUTPUT

```
Merge keys are not unique in right dataset; not a one-to-one merge
```

The endpoint repeated one record across a page boundary, and the declared join catches it.

The Join After the Repeat Is Dropped

PYTHON

```
stats[stats.duplicated("listing_id", keep=False)]
```

OUTPUT

```
     listing_id   seller agency  days_listed  views
149        3673  private    NaN            6     43
150        3673  private    NaN            6     43
```

PYTHON

```
stats = stats.drop_duplicates()
df = listings.merge(stats, on="listing_id", how="left", validate="one_to_one")
df.shape
```

OUTPUT

```
(180, 15)
```

NOTEBOOK · SECTIONS 4 TO 7

Pagination, Rendering, the Endpoint, and the Join

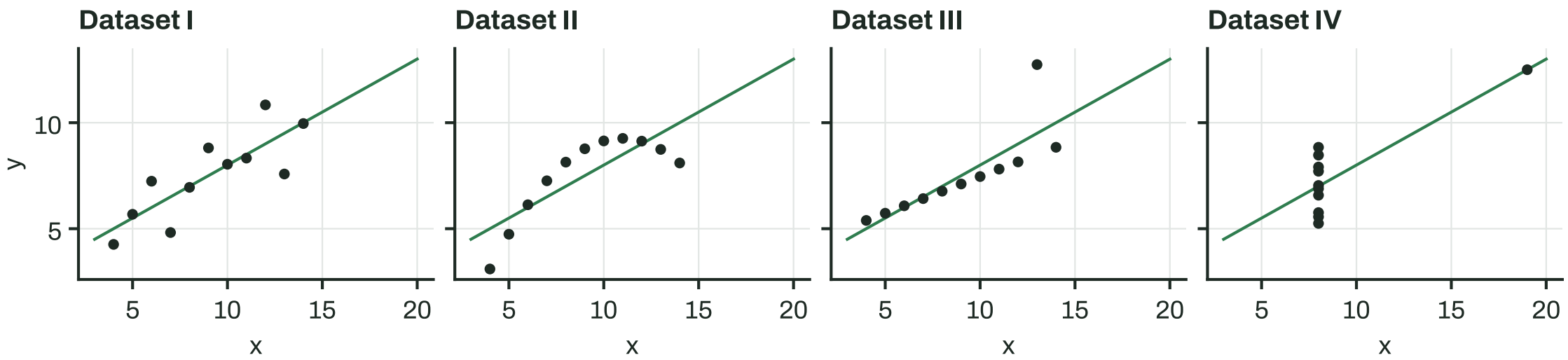
Run the cells from `### 4. Pagination` to `### 7. Joining Sources`.

02

Describing Data

Types, missingness, centre and spread, shape, counts, relationships, and groups

Anscombe's Quartet



The four samples share their means, their standard deviations, and a correlation of 0.82, yet they draw four different pictures. Everything measured is also drawn.

Variable Types

Type	Columns	Meaningful Operations
Quantitative, continuous	`price`, `area`	Every arithmetic summary is meaningful.
Quantitative, discrete	`rooms`, `floor`, `year_built`, `views`	Counts and arithmetic are meaningful, and the values are separate steps.
Categorical, nominal	`municipality`, `currency`, `seller`	Only counts, shares, and the mode are meaningful.
Categorical, ordinal	`condition`	Order, the median, and the mode are meaningful, but distances are not.

Labels as Categories

PYTHON

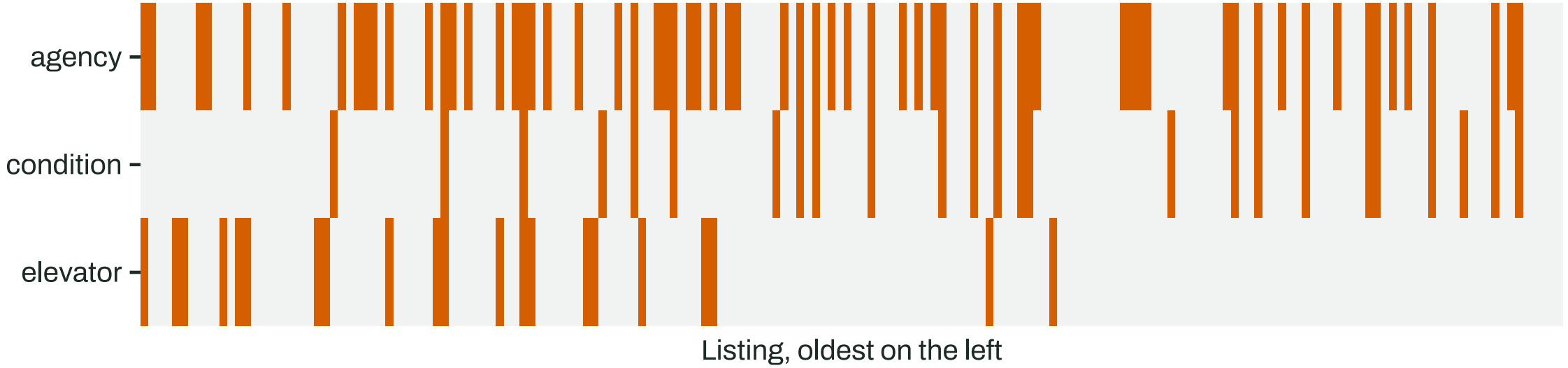
```python
for column in ["municipality", "currency", "seller", "agency"]:
    df[column] = df[column].astype("category")
df["condition"] = pd.Categorical(
    df["condition"], ordered=True,
    categories=["needs renovation", "good", "renovated", "new build"])
df["elevator"] = df["elevator"].astype("boolean")
df["condition"].min(), (df["condition"] >= "renovated").sum()
```

OUTPUT

```
('needs renovation', np.int64(46))
```

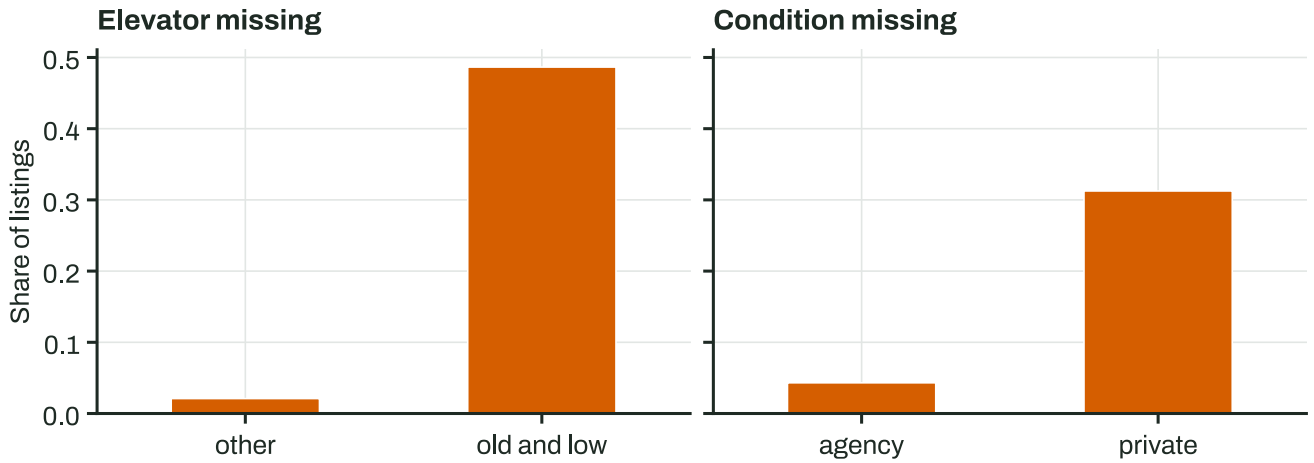
An ordered category sorts and compares by meaning rather than alphabetically.

The Shape of the Missingness



`agency` is missing for every private seller, because the field does not apply to them. `elevator` gaps gather among the oldest buildings on the left.

Crossing the Gaps With Other Columns



PYTHON

```python
old_and_low = ((df["year_built"] < 1980)
               & (df["floors"] <= 4))
gaps = df["elevator"].isna()
gaps.groupby(old_and_low).mean().round(2)
```

OUTPUT

```
False    0.02
True     0.49
Name: elevator, dtype: float64
```

A gap that depends on another column is not random, and its treatment depends on that reason.

Measures of Centre

A sample $x_1, \ldots, x_n$ sorts into $x_{(1)} \leq \cdots \leq x_{(n)}$.

Measure	Definition
Mean	$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$
Median	$x_{((n+1)/2)}$ for odd n, and $\frac{1}{2}\left(x_{(n/2)} + x_{(n/2+1)}\right)$ for even n.
Mode	It is the most frequent value.

The mode is the only measure of centre for a nominal variable.

Measures of Spread

Measure	Definition
Variance	$s_x^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$
Standard deviation	$s_x = \sqrt{s_x^2}$
Range	$x_{(n)} - x_{(1)}$
Interquartile range	$Q_3 - Q_1$

Q_1 and Q_3 are the first and third quartiles, so the interquartile range spans the middle half of the sample.

Area and Views

The Summaries

PYTHON

```python
summary = df[["area", "views"]].agg(
    ["mean", "median", "std"])
q1, q3 = df[["area", "views"]].quantile(
    [0.25, 0.75]).to_numpy()
summary.loc["IQR"] = q3 - q1
summary.round(1)
```

OUTPUT

	area	views
mean	68.4	845.6
median	65.8	439.5
std	22.2	1025.7
IQR	33.2	843.2

The Modes

PYTHON

```python
df["rooms"].mode().iloc[0]
```

OUTPUT

```
np.int64(2)
```

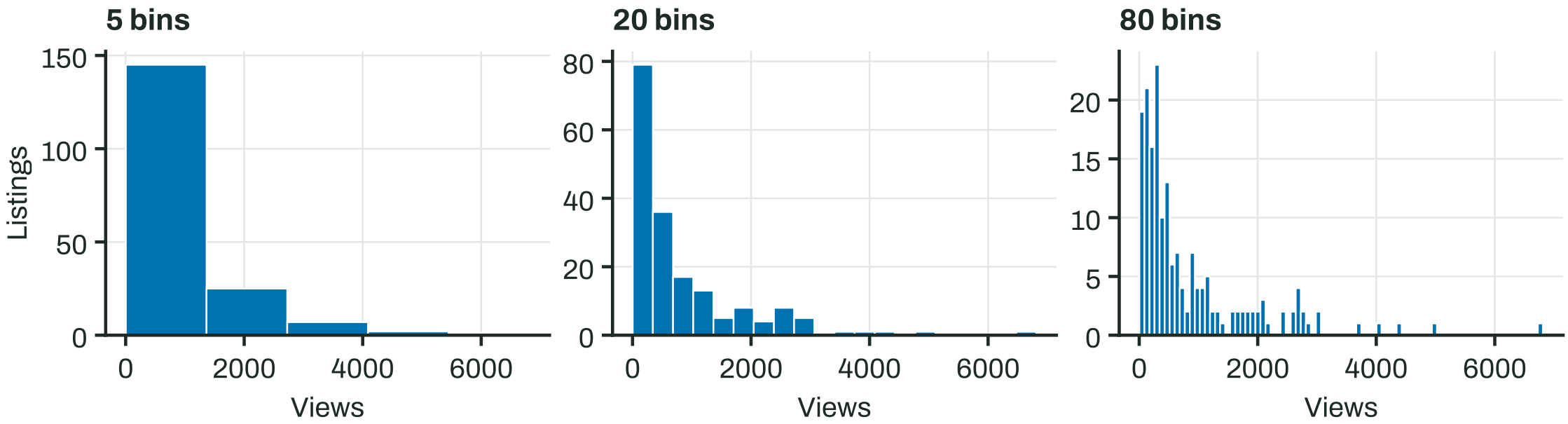
PYTHON

```python
df["municipality"].mode().iloc[0]
```

OUTPUT

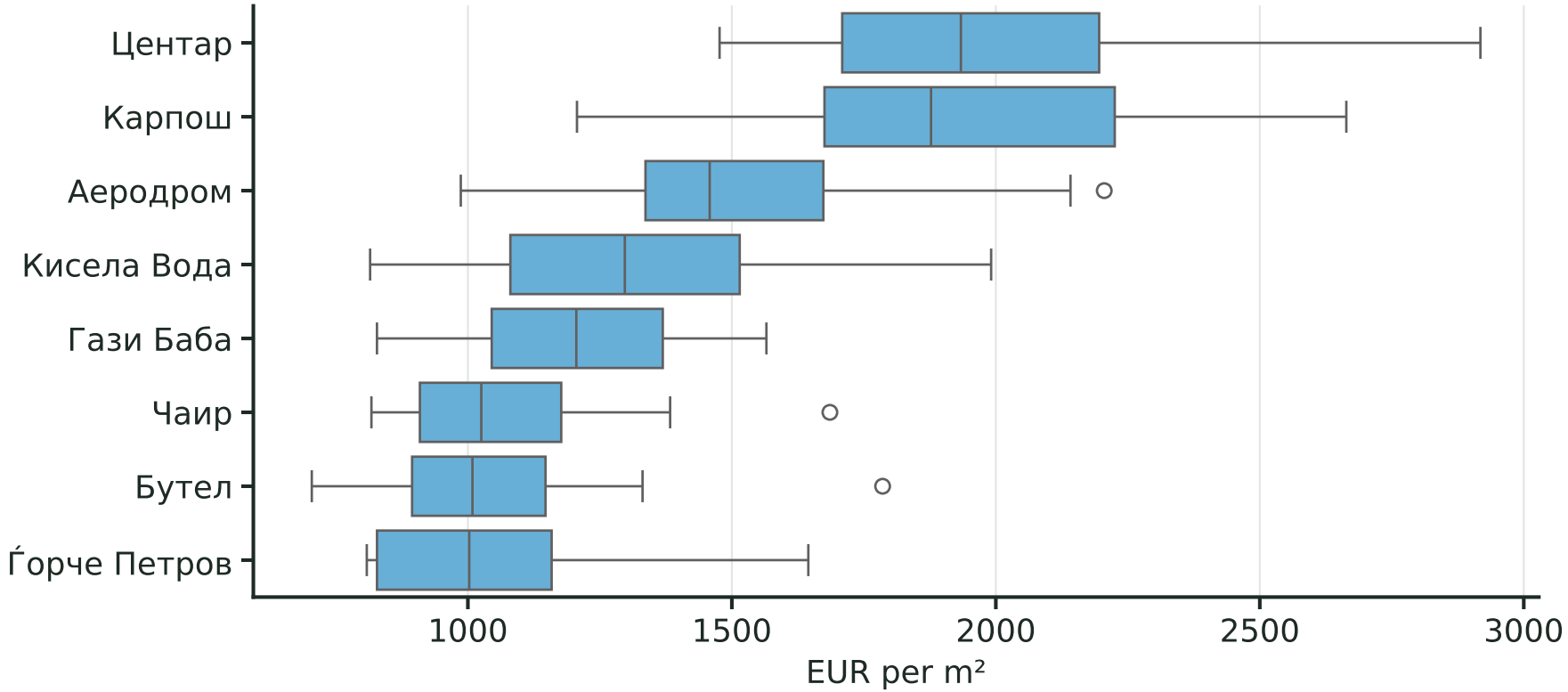
```
'Карпош'
```

Skew and the Bin Count



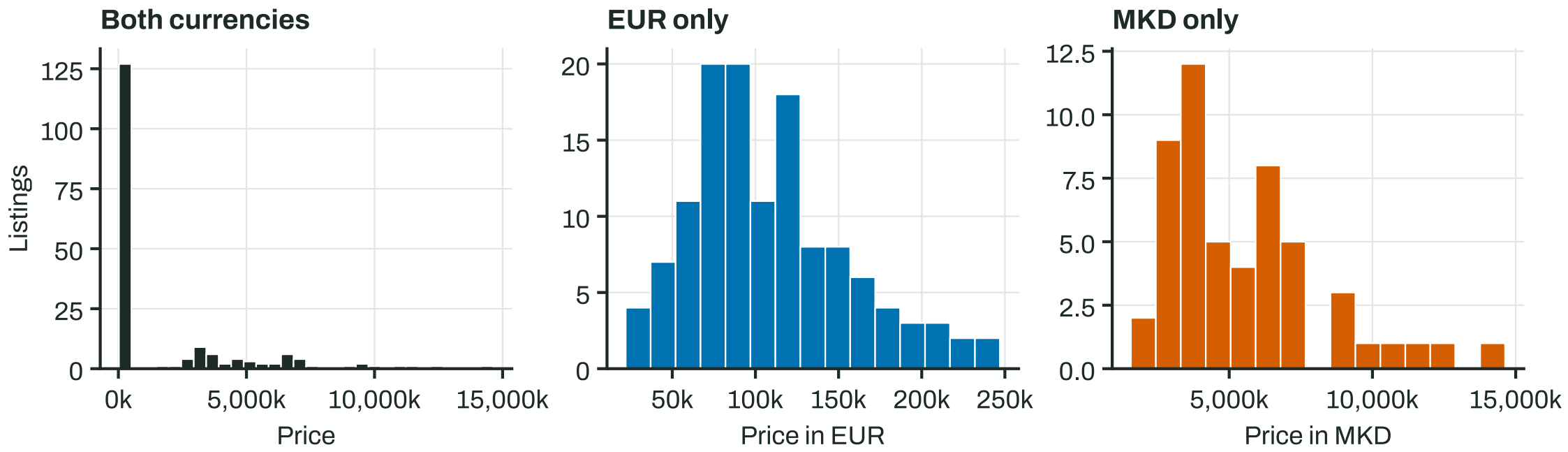
The mean of `views` lies far above its median, and the histogram shows the long right tail. Too few bins hide the shape, and too many turn it into noise.

The Box Plot



The box spans the interquartile range, the line inside it is the median, and a point beyond a whisker lies more than 1.5 interquartile ranges from the box.

A Column That Mixes Units



Pooled, the column is strongly skewed, because the MKD prices form a second cluster far to the right of the EUR prices. Within each currency the skew is smaller, so much of the pooled skew comes from mixing the units rather than from the prices themselves.

Comparing Within the Groups

PYTHON

```
df.groupby("currency", observed=True)["price"].agg(["count", "median", "skew"]).round(2)
```

OUTPUT

```
          count     median  skew
currency
EUR         127   100000.0  0.67
MKD          53  4830000.0  1.20
```

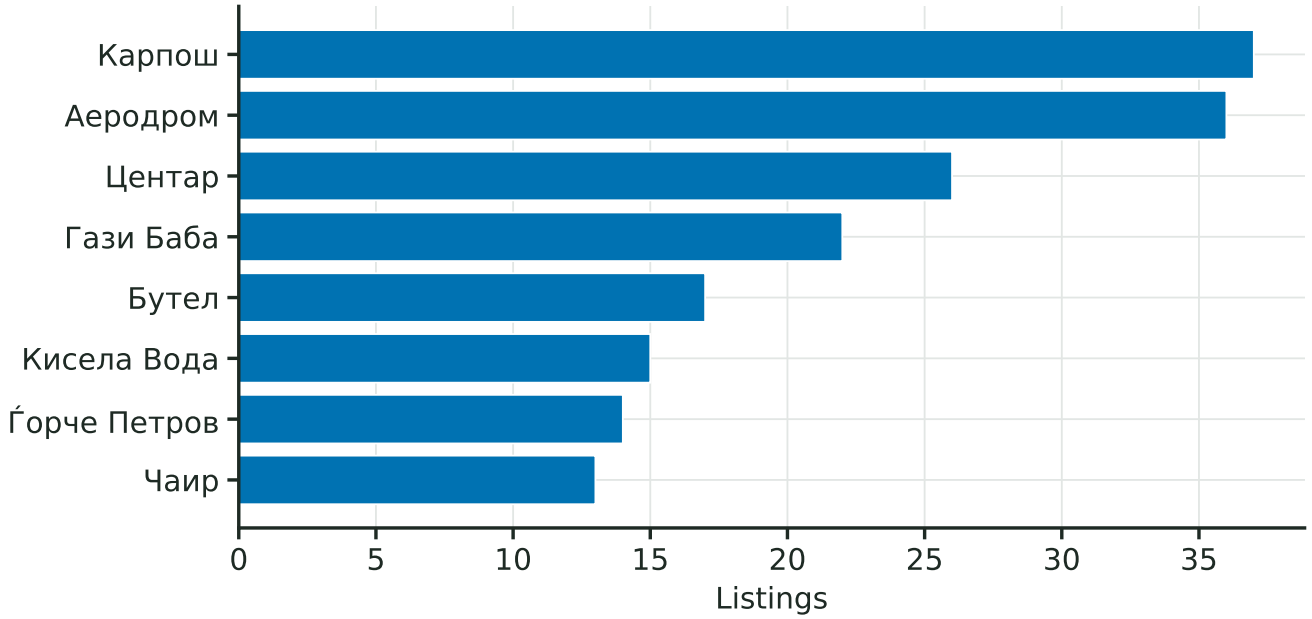
PYTHON

```
MKD_PER_EUR = 61.5
df["price_eur"] = df["price"].where(df["currency"] == "EUR", df["price"] / MKD_PER_EUR)
df["eur_per_m2"] = df["price_eur"] / df["area"]
df["eur_per_m2"].agg(["median", "skew"]).round(2)
```

OUTPUT

```
median    1467.38
skew         0.45
Name: eur_per_m2, dtype: float64
```

Counts of a Category



PYTHON

```python
pd.DataFrame({
    "count": df["municipality"].value_counts(),
    "share": df["municipality"].value_counts(
        normalize=True).round(3),
}).head(4)
```

OUTPUT

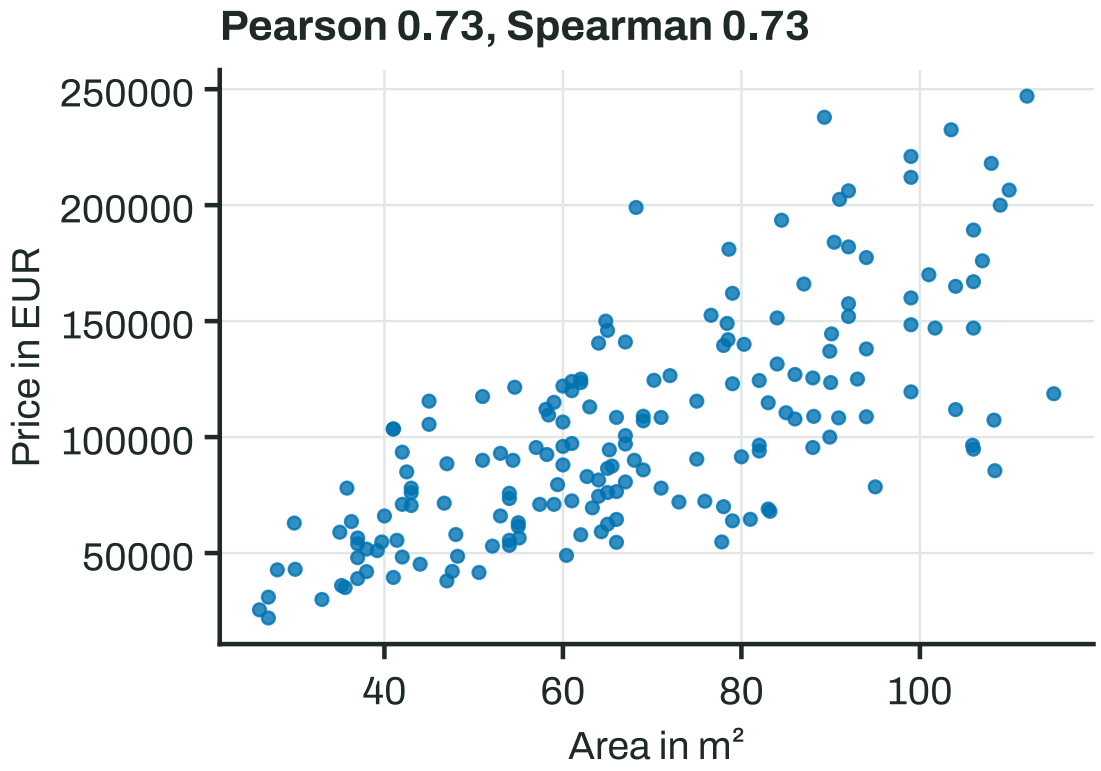
municipality	count	share
Карпош	37	0.206
Аеродром	36	0.200
Центар	26	0.144
Гази Баба	22	0.122

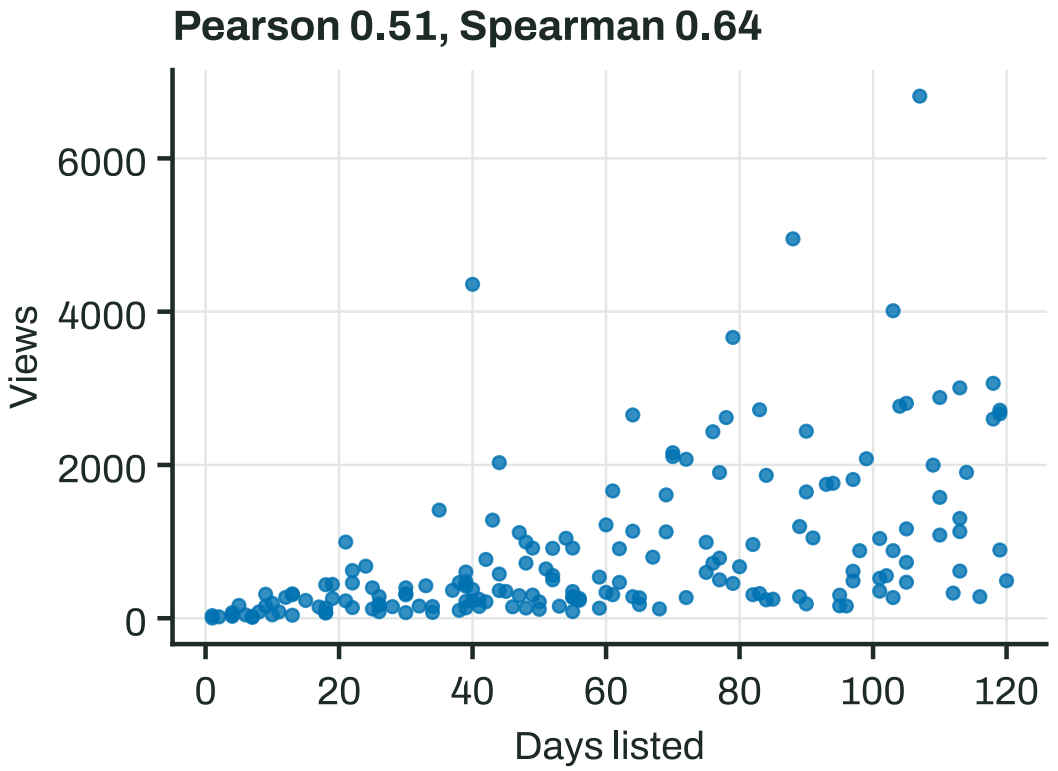
Pearson and Spearman

Measure	Definition
Covariance	$s_{xy} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$
Pearson	$r_{xy} = \frac{s_{xy}}{s_x\, s_y}$
Spearman	$\rho_{xy} = r_{uv}$, with $u_i = \mathrm{rank}(x_i)$ and $v_i = \mathrm{rank}(y_i)$.

Pearson's r measures how close the points lie to a straight line. Spearman's ρ is Pearson's r computed on the ranks, so it measures how close the points lie to a monotonic order and is less sensitive to a long tail.

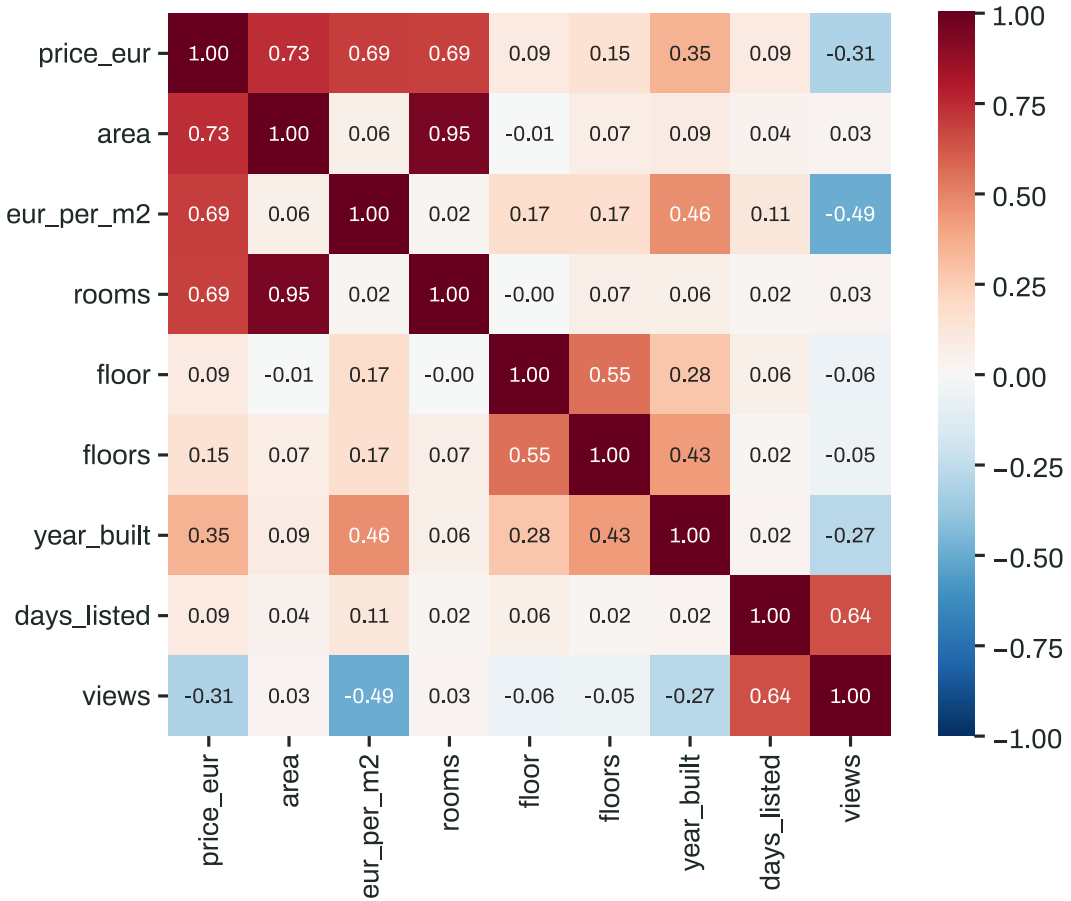
Relationships





Views and days listed correlate because views accumulate while a listing stays online. The long tail of `views` lowers Pearson's coefficient more than Spearman's.

The Correlation Heatmap



`DataFrame.corr(method="spearman")` gives the matrix. Correlation is not causation. A correlation says that two columns move together, but not which one moves the other, or whether a third factor moves both. The negative correlation between views and the price per square metre fits the explanation that buyers look more at cheaper flats, but the data alone cannot confirm that explanation or rule out another one.

Grouping and Aggregation

PYTHON

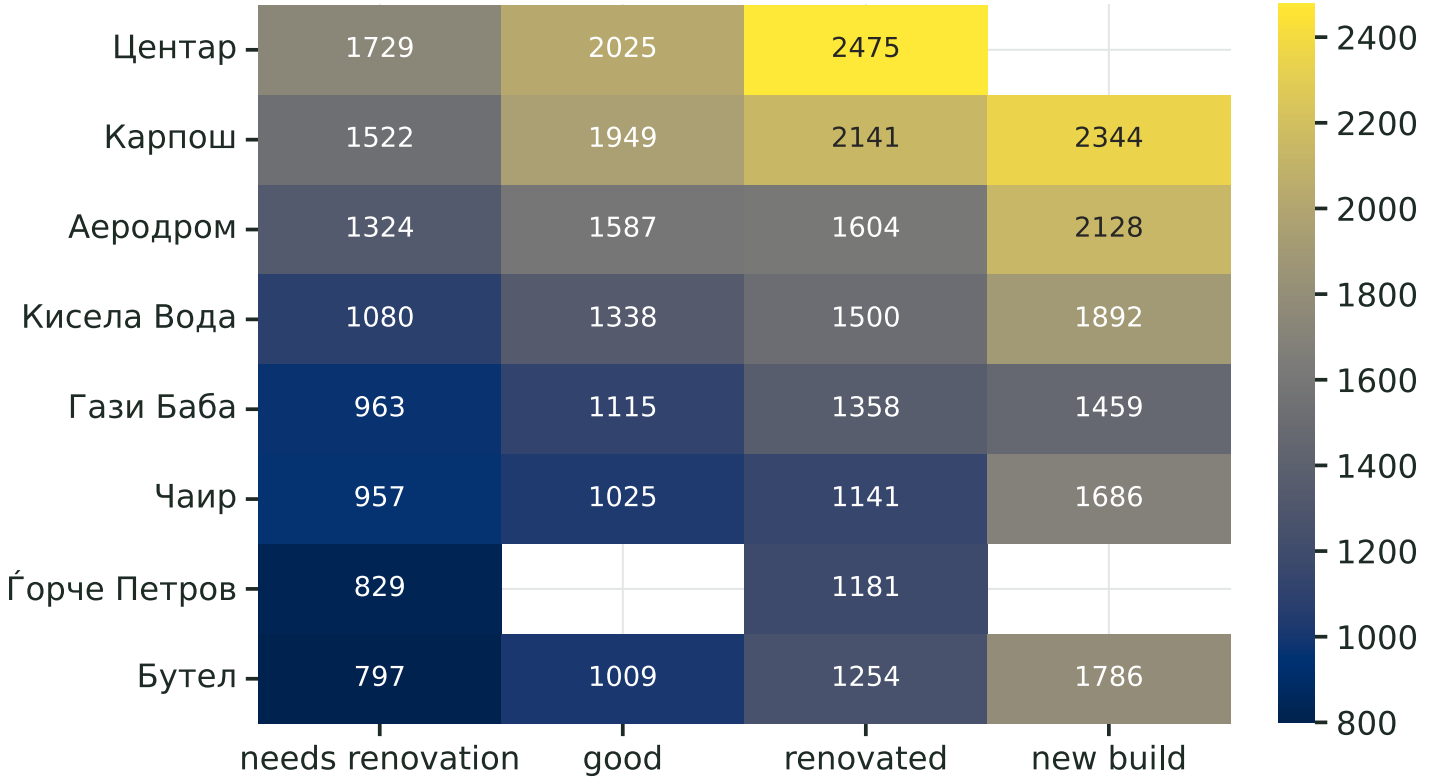
```python
(df.groupby("municipality", observed=True)
   .agg(listings=("listing_id", "size"),
        mean=("eur_per_m2", "mean"), median=("eur_per_m2", "median"))
   .sort_values("mean", ascending=False).round(0).head(3))
```

OUTPUT

municipality	listings	mean	median
Центар	26	1977.0	1934.0
Карпош	37	1916.0	1877.0
Аеродром	36	1530.0	1458.0

The mean reproduces the portal's published index, and the median lies below it in every municipality.

A Pivot Table



`pivot_table` crosses two labels and aggregates a value in each cell. A cell built from one or two listings is as noisy as its count suggests.

NOTEBOOK · SECTIONS 8 TO 16

Describing the Listings

Run the cells from `### 8. Anscombe's Quartet` to the end.



Reference

Variants, alternatives, and parameters for the tasks of this part

Requests: Parameters

Option	When It Is the Right Choice
`timeout=10`	It belongs on every call, because a request without it can wait forever.
`params={"page": 2}`	It builds a query string, and it is safer than formatting the address by hand.
`headers={"User-Agent": ...}`	It identifies the client when a server expects one.

Requests: The Response

Option	When It Is the Right Choice
`response.text`	It is the decoded body, for markup and other text.
`response.content`	It is the raw bytes, for files and for a parser that detects the encoding itself.
`response.json()`	It decodes a JSON body into dictionaries and lists.
`raise_for_status()`	It turns an error code into an exception, so a failed page cannot pass as data.

CSS Selectors

Selector	When It Is the Right Choice
`article`	It matches every element with the tag, when the tag alone identifies the item.
`.listing`	It matches a class, which is the usual hook for repeated items.
`#index`	It matches the one element with the identifier.
`article .price`	It matches a descendant at any depth inside the first match.

More CSS Selectors

Selector	When It Is the Right Choice
`ul > li`	It matches a direct child only, when deeper matches would be wrong.
`[data-id]`, `[data-id="2563"]`	It matches an attribute, or an attribute with a value.
`li:nth-of-type(2)`	It matches by position, when the markup has no class to use.
`tr:has(td)`	It matches an element by what it contains, such as a row with cells.

More Selectors in Use

PYTHON

```
{
    "attribute": len(soup.select("article[data-id]")),
    "child": len(soup.select("ul.features > li.rooms")),
    "position": soup.select_one("article:nth-of-type(2)")["data-id"],
    "contains": len(soup.select("article:has(.elevator)")),
    "attribute value": soup.select_one("article").get("data-agency"),
}
```

OUTPUT

```
{'attribute': 12, 'child': 12, 'position': '4159', 'contains': 12, 'attribute value': None}
```

`element["name"]` raises `KeyError` for an absent attribute, and `element.get("name")` returns `None`.

Regular Expression Constructs

Construct	When It Is the Right Choice
`\d`, `\s`, `\w`, `.`	They match one digit, one space, one word character, or any character.
`+`, `*`, `?`, `{4}`	They repeat the previous item one or more times, any number of times, at most once, or exactly four times.
`( ... )`, `(?: ... )`	The first captures a group, and the second groups without capturing.
`(?P<year> ... )`	It names a group, which reads better than a position when there are several.

Regular Expression Functions

Construct	When It Is the Right Choice
`re.search`	It finds the first match anywhere in the text.
`re.fullmatch`	It requires the whole text to match, for validating a value.
`re.findall`	It returns every match, for a sentence with several numbers.
`Series.str.extract`	It applies a pattern with groups to a whole column at once.

Regular Expressions in Use

PYTHON

```
sentence = "A 41.4 m² apartment on floor 5 of 8, built in 2011."
re.findall(r"\d+(?:\.\d+)?", sentence), re.search(r"built in (?P<year>\d{4})", sentence)["year"]
```

OUTPUT

```
(['41.4', '5', '8', '2011'], '2011')
```

PYTHON

```
summaries = pd.Series([c.select_one(".summary").get_text() for c in cards])
summaries.str.extract(r"floor (?P<floor>\d+) of (?P<floors>\d+)").head(3)
```

OUTPUT

```
  floor floors
0     6      8
1     1     10
2     5      8
```

Playwright

Option	When It Is the Right Choice
`async_playwright` with `await`	It is the interface inside a notebook, which already runs an event loop.
`sync_playwright`	It is the interface in a plain script, where no event loop runs.
`wait_for_selector("#index td")`	It waits for the element the code needs, which is the most reliable wait.
`wait_for_load_state("networkidle")`	It waits until the page stops loading resources, when no single element marks readiness.
`page.content()`	It returns the rendered markup for Beautiful Soup.

Pagination Patterns

Option	When It Is the Right Choice
A next link	It is followed until a page has none, when the pages link to each other.
A page number with a total	It is increased until the records account for the total, when an endpoint reports one.
An offset and a limit	The offset grows by the limit until a request returns fewer records than the limit.
A cursor	The client sends back the cursor of the last response until a response carries none.

Joining Frames

Option	When It Is the Right Choice
`how="inner"`	It keeps only the keys present on both sides.
`how="left"`	It keeps every row of the left frame, with missing values where the right has no match.
`how="outer"`	It keeps every key from both sides, for finding what each side lacks.

Validating and Stacking

Option	When It Is the Right Choice
`validate="one_to_one"`	It raises an error when either side repeats a key.
`validate="many_to_one"`	It allows repeated keys on the left and raises an error on a repeat on the right, for a lookup table.
`indicator=True`	It adds a `_merge` column that says where each row came from.
`pd.concat`	It stacks frames with the same columns, such as the pages of one source.

Types in pandas

Type	When It Is the Right Choice
`int64`, `float64`	They hold quantities, and `float64` also holds missing values as `NaN`.
`"category"`	It holds a nominal label with few distinct values, and it saves memory.
`pd.Categorical(..., ordered=True)`	It holds an ordinal label, so that sorting and comparisons follow the order.
`"boolean"`	It holds a yes or no value that can also be missing.
`observed=True`	It limits a grouping by a category to the labels that occur in the data.

Describing a Frame

PYTHON

```python
df[["area", "views"]].describe().round(1)
```

OUTPUT

```
          area    views
count   180.0    180.0
mean     68.4    845.6
std      22.2   1025.7
min      26.0      3.0
25%      52.8    214.0
50%      65.8    439.5
75%      86.0   1057.2
max     115.0   6812.0
```

Choosing a Summary of the Centre

Summary	When It Is the Right Choice
Mean and standard deviation	They suit a roughly symmetric quantity without extreme values.
Median and interquartile range	They suit a skewed quantity or one with outliers, because extreme values barely move them.
Mode	It is the centre of a nominal variable, and a useful extra for a discrete one.

Choosing a Summary of the Spread and Shape

Summary	When It Is the Right Choice
Range	It shows the extremes, and one outlier decides it.
Variance	It is the squared spread, which later formulas use, in squared units.
`Series.skew()`	It measures asymmetry: positive for a long right tail, and near zero for symmetry.
Quartiles with `quantile`	They give the five-number summary that the box plot draws.

Choosing a Plot

Plot	When It Is the Right Choice
Histogram, `sns.histplot`	It shows the distribution of one quantitative variable.
Bar plot, `sns.countplot`	It shows the count of each label of a categorical variable.
Box plot, `sns.boxplot`	It compares the distribution of a quantity across groups.
Scatter plot, `sns.scatterplot`	It shows the relationship between two quantitative variables.
Heatmap, `sns.heatmap`	It shows a matrix, such as correlations or a pivot table.
`hue=`, `order=`, `log_scale=`	They split by a label, fix the order of the labels, and draw a wide range on a log axis.

Choosing the Bin Count

PYTHON

```python
import numpy as np

{rule: len(np.histogram_bin_edges(df["views"], bins=rule)) - 1
 for rule in ["sturges", "fd", "auto"]}
```

OUTPUT

```
{'sturges': 9, 'fd': 23, 'auto': 23}
```

Rule	When It Is the Right Choice
`"sturges"`	It suits a small, roughly normal sample.
`"fd"`	It suits a skewed sample, because it uses the interquartile range.
`"auto"`	It takes the larger count of the two, and it is a reasonable default.

Correlation and Aggregation

Option	When It Is the Right Choice
`corr(method="pearson")`	It suits a linear relationship without a long tail.
`corr(method="spearman")`	It suits a monotonic relationship, a long tail, or ordinal data.
`groupby(...).agg(name=(column, function))`	It names each result column of an aggregation.
`size` and `count`	The first counts rows, and the second counts values that are not missing.

Reshaping Group Results

Option	When It Is the Right Choice
`groupby(...).transform`	It returns a value per row, such as the median of the row's group.
`pivot_table`	It crosses two labels and aggregates a value in each cell.
`pd.crosstab`	It counts the combinations of two labels, optionally as shares.
`sort_values` , `nlargest`	They order a result, or keep its top rows.

Grouping in Use

PYTHON

```python
median = df.groupby("municipality", observed=True)["eur_per_m2"].transform("median")
df.assign(relative=df["eur_per_m2"] / median).nlargest(3, "relative")[
    ["listing_id", "municipality", "eur_per_m2", "relative"]].round({"eur_per_m2": 0, "relative": 2})
```

OUTPUT

	listing_id	municipality	eur_per_m2	relative
48	4201	Бутел	1786.0	1.77
25	9674	Чаир	1686.0	1.64
85	4634	Ѓорче Петров	1645.0	1.64

Shares With a Cross Table

PYTHON

```python
pd.crosstab(df["municipality"], df["seller"], normalize="index").round(2).head(3)
```

OUTPUT

```
seller         agency  private
municipality
Ѓорче Петров     0.64     0.36
Аеродром         0.58     0.42
Бутел            0.82     0.18
```